计算机系统原理实验报告

第二次实验
NEMU 模拟器及 Cache 行为仿真

成员：	江李阳	202400460042
	王健一	202400460037
	邱泽辉	202400460046
	史淇	202333180074
专业：	密码科学与技术	
日期：	April 30, 2026	

Contents

1 实验目的 **3**

2 实验环境 **3**

3 实验原理 **3**
- 3.1 局部性原理 (Principle of Locality) . . . 3
- 3.2 Cache 结构与地址拆分 . . . 3
- 3.3 Cache 块映射方式 (Address Mapping) . . . 4
- 3.4 替换算法 (Replacement Algorithms) . . . 4
- 3.5 Cache 写策略 (Writing Policy) . . . 5

4 实验过程 **5**
- 4.1 NEMU 获取内存访问序列 . . . 6
  - 4.1.1 AM 环境下编译 workload . . . 6
  - 4.1.2 编译并配置 NEMU 模拟器 . . . 7
  - 4.1.3 运行程序并生成 trace.log . . . 7
- 4.2 Cache 模拟器设计与实现 . . . 7
  - 4.2.1 模拟器输入与输出 . . . 7
  - 4.2.2 Cache 参数设计 . . . 8
  - 4.2.3 地址拆分与映射过程 . . . 8
  - 4.2.4 命中检查机制 . . . 9
  - 4.2.5 替换算法实现 . . . 9
  - 4.2.6 核心访问流程 . . . 9
- 4.3 Cache 模拟器正确性验证 . . . 10
  - 4.3.1 测试一：重复访问同一地址 . . . 10
  - 4.3.2 测试二：访问同一 Cache 块内的不同地址 . . . 10
  - 4.3.3 测试三：直接映射冲突 . . . 11
  - 4.3.4 测试四：LRU 替换策略验证 . . . 11
  - 4.3.5 验证结果汇总 . . . 12
- 4.4 Cache 大小对命中率的影响 . . . 12
  - 4.4.1 实验结果 . . . 12
  - 4.4.2 实验现象描述与分析 . . . 14
  - 4.4.3 实验总结 . . . 15
- 4.5 相联度对 Cache 命中率的影响 . . . 15
  - 4.5.1 实验设计 . . . 15
  - 4.5.2 实验结果 . . . 16
  - 4.5.3 实验现象与分析 . . . 16
  - 4.5.4 实验总结 . . . 17
- 4.6 替换算法对 Cache 命中率的影响 . . . 17
  - 4.6.1 实验设计 . . . 17
  - 4.6.2 实验结果 . . . 18
  - 4.6.3 实验现象与分析 . . . 18
  - 4.6.4 实验总结 . . . 19
- 4.7 矩阵规模对 Cache 命中率的影响 . . . 19
  - 4.7.1 实验设计 . . . 19

4.7.2 不同矩阵规模下的工作集大小 20
4.7.3 实验结果 20
4.7.4 实验现象与分析 21
4.7.5 实验总结 21
4.8 矩阵乘法算法对 Cache 命中率的影响 22
4.8.1 实验设计 22
4.8.2 普通矩阵乘法的访存特征 22
4.8.3 分块矩阵乘法的优化思想 23
4.8.4 实验结果 24
4.8.5 实验现象与分析 24
4.8.6 实验总结 25
4.9 香山 Cache 性能计数器统计与对比（选做） 25
4.9.1 实验原理 25
4.9.2 实验流程 25
4.9.3 性能计数器解析方法 26
4.9.4 实验结果 29
4.9.5 结果分析 30
4.9.6 与软件 Cache 模拟器结果的对比 30
4.9.7 选做实验总结 30

5 实验总结 30

A Cache 模拟器代码 32

B 分析相联度对 Cache 命中率的影响代码 38

C 分析替换算法对 Cache 命中率的影响代码 40

D 分析矩阵规模对 Cache 命中率的影响代码 41

E 分析矩阵乘法算法对 Cache 命中率的影响代码 42

1 实验目的

学习 NEMU 模拟器的基本功能, 理解 cache 的工作原理, 以及相应参数与算法对程序性能的影响。

2 实验环境

- 硬件环境:
  - 处理器:13th Gen Intel(R) Core(TM) i5-13420H(12CPUs), 2.1GHz
  - 内存:16384MB RAM
- 软件环境:
  - 操作系统:Windows11
  - VMware Workstation Pro 17:Ubuntu 22.04 LTS 系统,7.8GB 内存,4 处理器,300GB 硬盘
  - Arch Linux x86_64(内核:Linux 6.6.87.2-microsoft-standard-WSL2)
    - L1d cache:288KiB(6 instances)
    - L1i cache:192 KiB(6 instances)
    - L2 cache:7.5 MiB(6 instances)
    - L3 cache:11MiB(1 instances)

3 实验原理

3.1 局部性原理 (Principle of Locality)

Cache 存在的理论基础就是局部性原理:

- 时间局部性 (Temporal Locality)：如果一个存储单元被访问，则近期内它很可能再次被访问（例如：循环变量、重复调用的函数）。
- 空间局部性 (Spatial Locality)：如果一个存储单元被访问，则其邻近的存储单元近期内也很可能被访问（例如：数组遍历、顺序执行的指令）。

Cache 通过存放这些“活跃”数据，减少了 CPU 访问高延迟 DRAM 的次数, 大大提高了系统运行效率。

3.2 Cache 结构与地址拆分

假设 Cache 的块的大小为 $B = 2^b$Byte, 共有 $S = 2^s$ 个组, 每组有 W 路, 则对于一个物理地址 Addr, 其逻辑地址被划分为三部分:

$$Addr = [\text{Tag}|\text{Index}|\text{Offset}]$$

- Offset(块内偏移):b 位, 决定数据在 Cache Line 中的具体字节位置。

- Index(组索引):s 位, 用于定位该地址属于哪一个 Set。计算公式为:

$$Index = (Addr >> b)\&(S-1)$$

- Tag(标志位): 剩余的高位, 用于区分映射到同一个组的不同内存块。

3.3 Cache 块映射方式 (Address Mapping)

Cache 地址映射方式决定了主存块如何被放置到 Cache 中, 是 Cache 实现的核心:

- 直接映射 (Direct Mapping)
  - 原理: 每个主存块映射到 Cache 中的唯一固定一行 (每组只有一行)。
  - 计算逻辑:$Cache$行号 $=$ (主存块号) (mod Cache 总行数)
  - 地址划分:
    - Offset: 块内偏移, 由块大小决定。
    - Index: 索引位: 直接对应 Cache 行号。
    - Tag: 高位剩余部分, 用于确定该行存放的是否是目标块。
  - 特点: 硬件电路最简单, 但冲突试效率高。如果两个经常访问的块映射到同一行, 会发生剧烈抖动。
- 全相联映射 (Fully Associative Mapping)
  - 原理: 主存中的任意一块可以放置在 Cache 中的任意一行 (因为只有一个组)。
  - 计算逻辑: 没有 Index 字段，只需匹配 Tag。
  - 地址划分: 只分为 Tag 和 Offset。
  - 特点: 冲突概率最低, 空间利用率最高。但因为全相联高速缓存电路必须并行地搜索许多相匹配的标记, 构造一个又大又快的相联高速缓存很困难, 而且很昂贵, 硬件成本极高, 功耗大, 通常只用作极小容量的存储 (如 TLB)。
- 组相联映射 (Set-Associative Mapping)
  - 原理: 这是前两者的折中。Cache 分为若干个组 (Set)，每组包含 k 个路 (Way)。主存块映射到固定的组，但可以放在该组内的任意一路。
  - 计算逻辑:$SetIndex =$ (主存块号) (mod $Cache$组数)
  - 特点: 兼顾了直接映射的低成本搜索和全相联的低冲突概率。实现时先通过组索引计算组号, 组中任何一行都可以包含任何映射到这一组的内存块, 所以必须搜索每组中每一行, 寻找一个有效的行, 其标记与地址中的标记相匹配。

3.4 替换算法 (Replacement Algorithms)

当 Cache 发生缺失 (Miss) 时, 且计算得到的 Cache 对应位置 (组) 已经满时, 必须决定换出哪一行。

- FIFO(First-In-First-Out, 先进先出)

  - 原理: 总是替换掉最先进入 Cache 的那一块。
  - 实现逻辑: 为每一行维护一个时间戳或使用一个循环队列, 最早调入的块被最先替换掉。
  - 评价: 实现简单, 但忽略了局部性原理, 因为一个很早被调入的块也可能是会被频繁访问的块, 这样一个频繁被访问的块也会被替换掉, 因此该算法性能一般。

- LRU(Least Recently Used, 最近最少使用)
  - 原理: 替换掉距离现在最长时间未被访问的那一块。
  - 实现逻辑: 每次命中某行时, 更新其计数器或移动到链表首部; 发生替换时选择计数器数值最小、反映" 最久远" 或链表尾部的行。
  - 评价:LRU 具体实现会较为复杂, 但能很好地利用局部性原理, 算法性能较优。

3.5 Cache 写策略 (Writing Policy)

Cache 的写操作分为写命中 (write hit) 和写不命中两种情况:

- 写命中:
  - 直写 (write-through): 立即将字 w 的告诉缓存块写回紧接的低一层, 实现简单, 但每次写都会引起总线流量。
  - 写回 (write-back): 尽可能迟地更新, 只有当替换算法将这个更新过的块要替换出去时才把它写到紧接着的低一层中。该策略利用了局部性原理, 能显著减少总线流量; 缺点是显著增加了复杂性。高速缓存必须为每一个高速缓存行维护一个额外的修改位 (dirty bit), 表明这个块是否被修改过。
- 写不命中:
  - 写分配 (write-allocate): 加载相应的低一层的块到 Cache 中并更新这个 Cache 块。该方法利用了写的空间局部性, 但缺点是每次写不命中都会导致一个块从低一层传送到 Cachhe。
  - 非写分配 (not-write-allocate): 避开 Cache, 直接将这个字写到低一层中。
- 直写 Cache 通常是非写分配的; 写回 Cache 通常是写分配的。

4 实验过程

本实验首先在香山仿真环境中基于 AM 编译示例程序，生成可在 NEMU 上运行的 RISC-V 裸机程序镜像；随后使用 NEMU 模拟器运行该程序，并通过开启内存访问日志功能获得程序执行过程中的内存地址访问序列，即 trace.log 文件。

在获得 trace.log 后，实验进一步编写 Cache 模拟器，以 trace.log 中记录的访存地址作为输入，按照给定的 Cache 容量、块大小、相联度和替换算法模拟 Cache 的访问过程，并统计总访问次数、命中次数、缺失次数以及命中率。

最后，通过改变 Cache 大小、Cache 块大小、相联度、替换算法、矩阵规模以及代码实现方式，观察不同配置下 Cache 命中率和 Miss 数的变化；在选做部分进一步使用香

山性能计数器统计真实处理器微结构下的 Cache 行为，并与自行编写的 Cache 模拟器结果进行对比分析。

本实验的总体流程如下：

1. 在 AM 环境中编译矩阵乘法程序，生成 RISC-V 可执行镜像；
2. 使用 NEMU 运行程序，并记录内存访问序列 trace.log；
3. 编写 Cache 模拟器，读取 trace.log 并模拟 Cache 命中与缺失过程；
4. 调整 Cache 参数和程序实现方式，统计不同配置下的 hit/miss 情况；
5. 根据实验数据分析 Cache 参数、替换算法和程序访存模式对命中率的影响。
6. 使用香山性能计数器统计 DCache 访存行为，并与软件 Cache 模拟器结果进行对比。

4.1 NEMU 获取内存访问序列

本实验首先需要获得待分析程序在运行过程中的内存访问序列。由于直接在真实硬件上观察每一次内存读写较为困难，因此实验使用 NEMU 模拟器运行矩阵乘法程序，并通过日志功能记录程序执行过程中的访存行为，生成 trace.log 文件。该文件随后作为 Cache 模拟器的输入，用于统计不同 Cache 配置下的命中次数和缺失次数。

4.1.1 AM 环境下编译 workload

香山实验环境部署完成后，首先进入 xs-env 目录并配置环境变量：

```
cd ~/xs-env
source ./env.sh
```

随后进入 AM 提供的 hello 示例程序目录，将其中的 hello.c 替换为本次实验使用的矩阵乘法程序：

```
cd ~/xs-env/nexus-am/apps/hello
make ARCH=riscv64-xs -j8
```

编译成功后，在 build 目录下可以得到如下文件：

- `hello-riscv64-xs.elf`：应用程序的 ELF 文件；
- `hello-riscv64-xs.bin`：可由 NEMU 加载运行的二进制镜像；
- `hello-riscv64-xs.txt`：对应的反汇编文件。

其中，后续实验主要使用 `hello-riscv64-xs.bin` 作为 NEMU 的运行输入。

4.1.2 编译并配置 NEMU 模拟器

进入 NEMU 目录后，首先进行基本配置和编译：

```
cd ~/xs-env/NEMU
make clean
make riscv64-xs_defconfig
make menuconfig
make -j
```

由于 NEMU 默认不一定开启访存日志功能，因此需要在 `make menuconfig` 中开启相关调试选项。具体路径为：

```
Testing and Debugging
[*] Enable Log for memory
[*] Enable Log for tracing instructions
```

其中，`Enable Log for memory` 用于记录内存读写行为，`Enable Log for tracing instructions` 用于记录指令执行过程。修改配置后，需要重新执行：

```
make -j
```

使新的配置生效。

4.1.3 运行程序并生成 trace.log

NEMU 编译完成后，使用如下命令运行矩阵乘法程序：

```
./build/riscv64-nemu-interpreter \
-b ~/xs-env/nexus-am/apps/hello/build/hello-riscv64-xs.bin \
-l trace.log
```

其中，`-b` 参数用于指定待运行的二进制镜像，`-l` 参数用于指定日志输出文件。程序运行结束后，NEMU 会在当前目录下生成 `trace.log` 文件，该文件记录了程序执行过程中的访存地址、访存类型等信息。

本实验分别修改矩阵规模，生成了矩阵大小为 $N = 32$、$N = 64$、$N = 128$ 时的三组 trace 文件，并将其作为后续 Cache 模拟器分析的输入。

4.2 Cache 模拟器设计与实现

在获得 NEMU 输出的内存访问序列 trace.log 后，本实验进一步编写 Cache 模拟器，对 trace 中记录的访存地址进行逐条模拟访问，从而统计不同 Cache 配置下的 Hit 次数、Miss 次数和命中率。

Cache 模拟器的核心目标并不是模拟完整处理器，而是抽象出 Cache 的基本工作机制，包括地址拆分、组索引计算、Tag 匹配、命中判断以及缺失后的替换过程。通过这种方式，可以在软件层面观察不同 Cache 参数对程序访存性能的影响。

4.2.1 模拟器输入与输出

Cache 模拟器的输入为 NEMU 生成的 trace.log 文件。该文件记录了程序运行过程中的访存行为，模拟器从中提取访存地址，并按照给定 Cache 配置依次进行访问模拟。

模拟器的输出主要包括：

- 总访存次数；
- Cache Hit 次数；
- Cache Miss 次数；
- Cache 命中率；
- 不同配置下的统计结果对比。

其中，命中率的计算公式为：

$$HitRate = \frac{Hit}{Hit + Miss} \times 100\%$$

4.2.2 Cache 参数设计

本实验中的 Cache 模拟器支持对以下参数进行调整：

- `cache_size`：Cache 总容量；
- `block_size`：Cache 块大小；
- `associativity`：相联度，即每组包含的 Cache Line 数；
- `replacement_policy`：替换算法，包括 FIFO 和 LRU；
- `set_num`：Cache 组数。

其中，Cache 组数由 Cache 总容量、块大小和相联度共同决定：

$$set_num = \frac{cache_size}{block_size \times associativity}$$

通过改变 `associativity` 的取值，可以统一表示不同的映射方式：

- 当 `associativity = 1` 时，Cache 为直接映射；
- 当 `associativity = k` 时，Cache 为 k 路组相联；
- 当 `set_num = 1` 时，Cache 为全相联映射。

4.2.3 地址拆分与映射过程

对于一条访存地址 `addr`，模拟器首先根据块大小计算其所属的主存块号：

$$block_addr = \left\lfloor \frac{addr}{block_size} \right\rfloor$$

随后根据 Cache 组数计算组索引：

$$set_index = block_addr \bmod set_num$$

最后计算 Tag：

$$tag = \left\lfloor \frac{block_addr}{set_num} \right\rfloor$$

因此，每一次访存都可以被映射到某一个 Cache Set 中。模拟器只需要在该组内查找是否存在相同 Tag 的有效 Cache Line，即可判断本次访问是否命中。

4.2.4 命中检查机制

模拟器处理每一条访存记录时，首先计算该地址对应的 `set_index` 和 `tag`，然后在对应的 Cache Set 中顺序查找：

1. 若存在 `valid = 1` 且 `tag` 相同的 Cache Line，则本次访问为 Hit；
2. 若不存在匹配项，则本次访问为 Miss；
3. 若发生 Miss 且该组仍有空闲 Cache Line，则直接装入新块；
4. 若发生 Miss 且该组已满，则根据替换算法选择一个 Cache Line 换出。

该过程体现了 Cache 的基本工作逻辑：先利用 Index 定位组，再在组内进行 Tag 匹配，最后根据匹配结果判断命中或缺失。

4.2.5 替换算法实现

当某个 Cache Set 已满且新的内存块需要调入时，模拟器需要根据替换算法选择被换出的 Cache Line。本实验主要实现了 FIFO 和 LRU 两种策略。

FIFO 替换算法 FIFO 即先进先出策略。该算法记录每个 Cache Line 被装入 Cache 的时间，当需要替换时，选择最早进入 Cache 的块换出。FIFO 的实现简单，但它并不关心 Cache Line 最近是否被访问过，因此可能会把仍然频繁使用的数据块换出。

LRU 替换算法 LRU 即最近最少使用策略。该算法记录每个 Cache Line 最近一次被访问的时间，当需要替换时，选择最久没有被访问的块换出。相比 FIFO，LRU 更符合时间局部性原理，通常能够更好地保留近期可能再次访问的数据。

4.2.6 核心访问流程

Cache 模拟器的核心访问逻辑可以概括如下：

Listing 1: Cache 访问模拟的核心逻辑

```
AccessCache(addr):
    block_addr = addr / block_size
    set_index  = block_addr % set_num
    tag        = block_addr / set_num

    for each line in cache[set_index]:
        if line.valid == true and line.tag == tag:
            hit++
```

```
            update_lru_information()
            return

    miss++

    if there is an empty line in cache[set_index]:
        fill the empty line with new tag
    else:
        victim = select_victim_by_policy()
        replace victim line with new tag
```

由此可见，Cache 模拟器的关键在于正确实现地址映射和替换逻辑。只要 trace.log 的访存地址能够被正确解析，模拟器就可以在不同 Cache 参数下重复运行，从而获得可比较的实验数据。

4.3 Cache 模拟器正确性验证

为了验证自行编写的 Cache 模拟器是否正确，本实验采用 micro-benchmarking 的方法，构造若干组规模较小、访问规律明确的访存序列，并人工推导其理论 Hit/Miss 结果，再与模拟器输出结果进行对比。

这种验证方法的优点是访存序列简单、预期结果明确，能够分别检查 Cache 模拟器中的块内偏移处理、地址映射、冲突缺失和替换算法等核心逻辑是否正确。

4.3.1 测试一：重复访问同一地址

首先构造最简单的重复访问序列，用于验证 Cache 的基本装入和命中判断逻辑。

- Cache 容量：64B；
- Cache 块大小：16B；
- 映射方式：直接映射；
- 访问序列：`0x00`，`0x00`，`0x00`。

第一次访问 `0x00` 时，Cache 中尚未装入对应数据块，因此发生 Miss；随后两次访问仍然访问同一地址，对应数据块已经在 Cache 中，因此均为 Hit。理论结果为：

$$Miss = 1, \quad Hit = 2$$

4.3.2 测试二：访问同一 Cache 块内的不同地址

第二组测试用于验证块大小和块内偏移处理是否正确。

- Cache 容量：64B；
- Cache 块大小：16B；
- 映射方式：直接映射；

- 访问序列：`0x00`，`0x04`，`0x08`，`0x0C`。

由于块大小为 16B，地址 `0x00`、`0x04`、`0x08`、`0x0C` 都位于同一个 Cache Block 内。因此第一次访问发生 Miss，后续三次访问均应命中。理论结果为：

$$Miss = 1, \quad Hit = 3$$

4.3.3 测试三：直接映射冲突

第三组测试用于验证直接映射下的冲突缺失处理是否正确。

- Cache 容量：64B；
- Cache 块大小：16B；
- Cache 行数：4；
- 映射方式：直接映射；
- 访问序列：`0x00`，`0x40`，`0x00`。

由于 Cache 容量为 64B，块大小为 16B，因此共有 4 个 Cache Line。地址 `0x00` 和 `0x40` 对应的主存块号分别为 0 和 4，二者对 Cache 行数取模后都映射到第 0 行。因此访问 `0x40` 会替换掉原先的 `0x00`，再次访问 `0x00` 时仍然发生 Miss。理论结果为：

$$Miss = 3, \quad Hit = 0$$

4.3.4 测试四：LRU 替换策略验证

第四组测试用于验证 LRU 替换算法是否正确。

- Cache 容量：32B；
- Cache 块大小：16B；
- 相联度：2 路组相联；
- Cache 组数：1；
- 替换算法：LRU；
- 访问序列：`0x00`，`0x10`，`0x00`，`0x20`。

该配置下 Cache 只有一个 Set，且该 Set 中有两个 Cache Line。首先访问 `0x00` 和 `0x10`，两者分别装入两个 Cache Line；随后再次访问 `0x00`，使得 `0x00` 成为最近使用的数据块；最后访问 `0x20` 时，该 Set 已满，根据 LRU 策略应替换最久未被访问的 `0x10`，而不是 `0x00`。

因此该访问序列的理论结果为：

$$Miss = 3, \quad Hit = 1$$

4.3.5 验证结果汇总

将上述测试序列输入 Cache 模拟器后，模拟器输出结果与人工推导结果一致，说明模拟器在基本命中判断、块内偏移处理、直接映射冲突处理以及 LRU 替换逻辑方面均能够正确工作。

Table 1: Cache 模拟器正确性验证结果

测试编号	测试目的	理论结果	验证结果
1	重复访问同一地址	Miss=1, Hit=2	一致
2	同一块内连续访问	Miss=1, Hit=3	一致
3	直接映射冲突	Miss=3, Hit=0	一致
4	LRU 替换策略	Miss=3, Hit=1	一致

通过以上 micro-benchmarking 验证，可以说明 Cache 模拟器的核心逻辑是正确的。随后再使用 NEMU 生成的大规模 trace.log 进行实验分析时，所得 Hit/Miss 统计结果具有较好的可信度。

4.4 Cache 大小对命中率的影响

4.4.1 实验结果

选取矩阵规模为 128 时的 trace 数据分析, 固定映射方法为 4 路组相联、替换算法为"LRU", 分别测试分组大小为 $16B, 64B, 128B$ 在 Cache 大小分别为 $1, 2, 4, 8, 16, 32, 64, 128KB$ 时的命中率, 结果如下: 对应折线图如图:

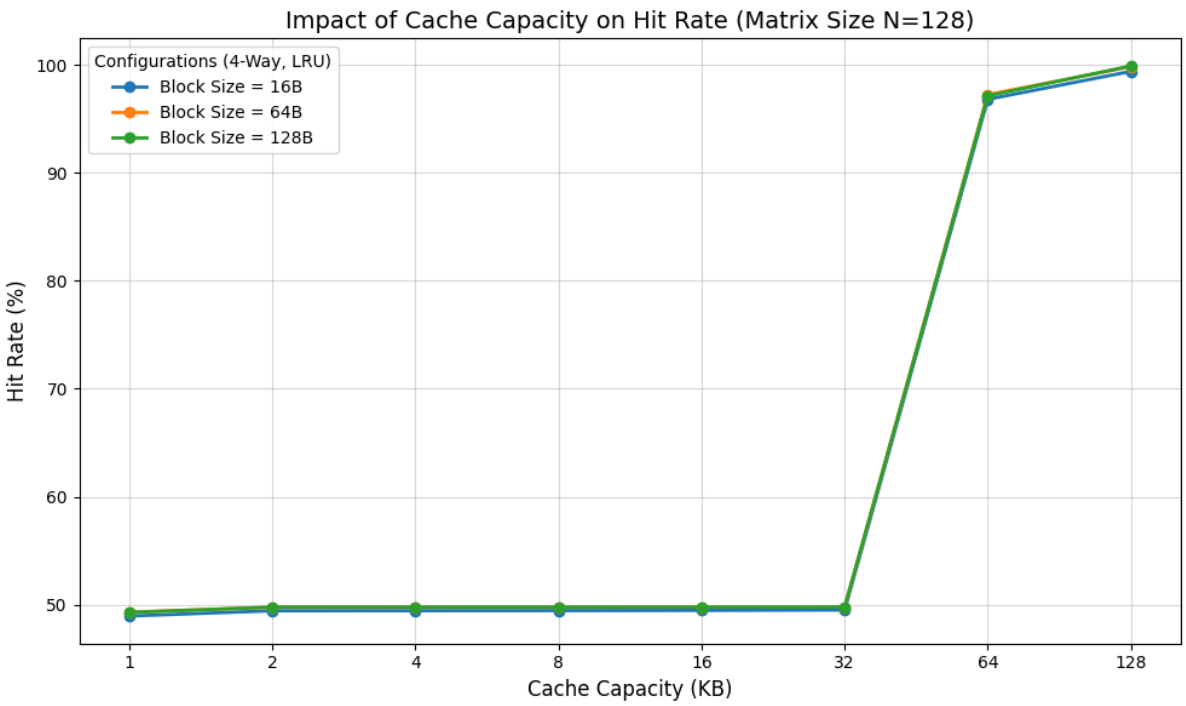

Figure 1: Cache 大小对命中率的影响

Table 2: Cache 大小对命中率的影响

Cache 大小 (KB)	块大小 (B)	命中率 (%)	Miss 数
1	16	48.934697591460555	2171337
2	16	49.42544576429553	2150470
4	16	49.43146634857913	2150214
8	16	49.43974465196907	2149862
16	16	49.457806404819856	2149094
32	16	49.493929910521416	2147558
64	16	96.83107957307472	134745
128	16	99.42044820898201	24643
1	64	49.29104092374577	2156185
2	64	49.74862884720627	2136728
4	64	49.75163913934807	2136600
8	64	49.75239171238352	2136568
16	64	49.75765972363166	2136344
32	64	49.76650245679819	2135968
64	64	97.26987668855635	116087
128	64	99.85480043997302	6174
1	128	49.2675700522027	2157183
2	128	49.76130499927212	2136189
4	128	49.76431529141392	2136061
8	128	49.76431529141392	2136061
16	128	49.76431529141392	2136061
32	128	49.77033587569751	2135805
64	128	97.14652526446474	121332
128	128	99.92725911254236	3093

4.4.2 实验现象描述与分析

- 数据基础: 在 $N = 128$ 的 int 型矩阵乘法中
  - 单行大小:$128 \times 4Bytes = 512Bytes$
  - 单个矩阵大小:$128 \times 128 \times 4Bytes = 64KB$
  - 三个矩阵总大小 (A,B,C):$64KB \times 3 = 192KB$
- 阶段性现象与原因分析:
  - 低容量阶段 ($1KB \sim 32KB$)
    - 现象描述: 命中率极低且极其稳定, 基本维持在 49% 左右, 即使 Cache 容量从 1KB 增加到 32KB(增加了 32 倍), 命中率几乎没有实质性提升,Miss 数居高不下。
    - 原因分析:
      - 数据量远超容量: 此时 Cache 容量最大仅为 $32KB$, 而单个矩阵大小为 $64KB$。这意味着 Cache 连一个完整的矩阵都无法容纳。
      - 矩阵 B 的破坏性访问: 矩阵乘法中,B 矩阵是按列访问的 ($B[k][j]$), 其访存步长为 512Bytes。在容量小于 $64KB$ 时, 每当外层循环切换，之前缓存在 Cache 中的 B 矩阵行数据会因为容量不足而被后续数据替换, 产生大量的 Cache Miss。
      - 命中率的构成：此阶段的命中主要来源于矩阵 A 的行优先访问 (利用了空间局部性) 和矩阵 C 的重复累加 (利用了时间局部性)，这两者约占总访存的一半，故命中率固定在 49% 附近。
  - 性能跃迁 ($32KB \rightarrow 64KB$)
    - 现象描述: 当容量从 $32KB$ 增加到 $64KB$ 时，命中率从 49.7% 瞬间飙升至 95% 以上, 从图像上看图线的斜率陡然增加,Cache 性能大幅上升。
    - 原因分析:
      - 数据容纳:$64KB$ 正好等于一个 int 型矩阵 ($128 \times 128 \times 4$) 的大小。当 Cache 容量达到 64KB 时, 系统恰好能够完整地缓存下矩阵 A 或矩阵 B 的绝大部分活跃数据。
      - 循环间访存失效消除: 此时, 在内层循环中被加载的 B 矩阵块可以驻留在 Cache 中供后续循环使用。这种从“无法容纳”到“完全容纳”的改变，消除了绝大部分的重复容量失效，导致了命中率的断崖式上升。
  - 饱和稳定阶段 ($64KB \sim 128KB$)
    - 现象描述: 命中率进一步小幅度提升并最终稳定在 99.8% 左右。
    - 原因分析:
      - 数据完全容纳: 当容量达到 128KB 时，已非常接近总数据量 ($192KB$) 的量级。此时除了最初加载数据产生的不可避免的冷启动失效 (Cold Miss) 外, 几乎所有的 Cache Miss 都消失了。
      - 边际效益递减：由于核心数据集已被完全容纳, 继续增加容量对命中率的提升空间极小, 性能达到硬件优化上限。

- 块大小 (Block Size) 现象分析:
  - 现象描述: 在数据表中，相同的容量下，改变 Block Size($16B, 64B, 128B$) 对结果的影响非常微弱, 基本可以忽略不计。
  - 原因分析: 矩阵 B 的访存步长为 512B, 这远大于我们测试的最大块大小 (128B)。因此, 增大块大小无法通过预取机制帮矩阵 B 找回数据，无法利用其空间局部性。

4.4.3 实验总结

通过对 int 矩阵的测试可以得出, 对于特定的算法和矩阵规模,Cache 大小存在性能阈值, 该阈值与单个矩阵大小有密切关系; 非优化的矩阵乘法由于存在跨行访存 (B 矩阵), 其对 Cache 块大小并不敏感, 性能的提升完全依赖于 Cache 总容量是否能覆盖 B 的活跃数据。

4.5 相联度对 Cache 命中率的影响

为了进一步分析 Cache 映射方式对命中率的影响，本实验在固定 Cache 容量、块大小和替换算法的条件下，改变 Cache 的相联度，比较直接映射、组相联映射和全相联映射在同一访存序列下的 Hit/Miss 情况。

本组实验采用矩阵规模为 $N = 128$ 时生成的 trace.log 作为输入，固定 Cache 块大小为 64B，替换算法为 LRU，并分别测试不同相联度下的 Cache 命中率。

4.5.1 实验设计

本实验固定如下参数：

- Cache 大小：64KB；
- 矩阵规模:$N = 128$;
- Cache 块大小:64B;
- 替换算法:LRU;
- 输入文件：$N = 128$ 时由 NEMU 生成的 trace.log。

改变的参数为 Cache 相联度，包括：

- 1 路组相联，即直接映射；
- 2 路组相联；
- 4 路组相联；
- 8 路组相联；
- 全相联映射。

其中，直接映射可以看作相联度为 1 的特殊组相联 Cache；全相联映射可以看作只有 1 个 Set、所有 Cache Line 都位于同一组内的特殊情况。

4.5.2 实验结果

Table 3: 相联度对 Cache 命中率的影响

Cache 大小	相联度	映射方式	命中率 (%)	Miss 数
64KB	1 路	直接映射	97.83	92177
64KB	2 路	2 路组相联	98.16	78164
64KB	4 路	4 路组相联	97.26	116087
64KB	8 路	8 路组相联	95.46	192655
64KB	全相联	全相联映射	96.79	136480

实验数据绘制如下柱状图:

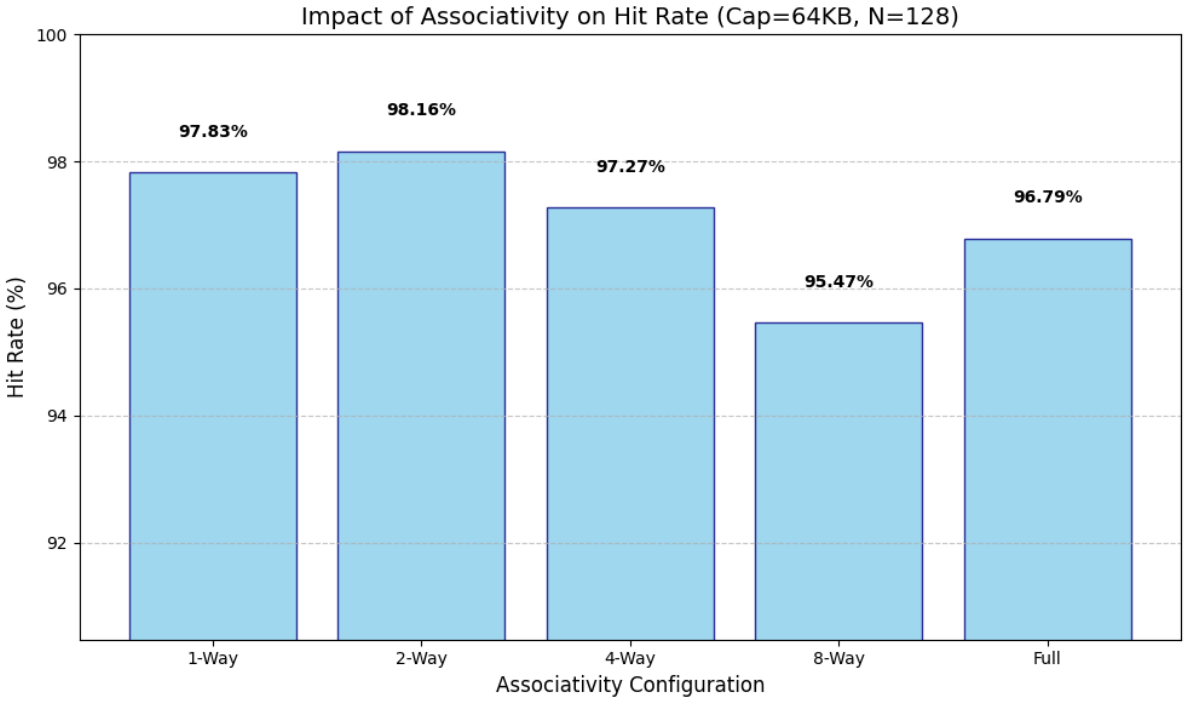

Figure 2: 相联度对 Cache 命中率的影响 (Cache capacity = 64KB,Cache block = 64B)

4.5.3 实验现象与分析

从实验结果可以看出，在固定 Cache 大小为 64KB、块大小为 64B、替换算法为 LRU 的条件下，Cache 命中率并没有随着相联度的增大而单调上升。具体而言，2 路组相联的命中率最高，为 98.16%，Miss 数为 78164；直接映射的命中率为 97.83%，略低于 2 路组相联；而 4 路组相联、8 路组相联和全相联的命中率分别下降到 97.26%、95.46% 和 96.79%。

这一现象说明，在本实验的访存序列和模拟器实现条件下，影响命中率的不仅是相联度本身，还包括 Cache 组数、地址分布、替换行为以及程序访存模式等因素。

理论上，提高相联度可以增加同一个 Set 内可容纳的 Cache Line 数量，从而缓解多个主存块映射到同一位置时产生的冲突缺失。但是在 Cache 总容量和块大小固定的情况下，提高相联度会减少 Cache 的组数。例如，当 Cache 容量固定为 64KB、块大小固定

为 64B 时，1 路、2 路、4 路和 8 路对应的组数依次减少。组数减少后，更多主存块会被映射到同一个 Set 中，使得该 Set 内部的替换压力增加。

因此，相联度提高带来的“每组可容纳更多块”的好处，可能会被“组数减少导致更多地址集中到同一组”的影响部分抵消。对于本实验中的矩阵乘法 trace，2 路组相联在冲突缓解和组数保留之间取得了较好的平衡，因此命中率最高；而 8 路组相联虽然每组容量更大，但组数更少，局部 Set 的竞争更加集中，导致 Miss 数反而增加。

全相联映射理论上可以消除组索引冲突，因为任意主存块都可以放入任意 Cache Line 中。但在实际模拟时，全相联也意味着所有块都在同一个集合中参与替换，替换算法对整体访问序列更加敏感。因此，本实验中全相联的命中率虽然高于 8 路组相联，但并没有超过 2 路组相联。

4.5.4 实验总结

通过本组实验可以看出，相联度对 Cache 命中率的影响不是简单的单调关系。理论上，提高相联度有助于减少冲突缺失；但在 Cache 总容量固定时，相联度提高会同时减少 Set 数量，使更多地址竞争同一组中的 Cache Line。因此，相联度过高不一定带来更高命中率。

本实验数据表明，2 路组相联在当前矩阵乘法 trace 下取得了最高命中率，说明适中的相联度能够在冲突缺失和组数规模之间取得较好平衡。实际 Cache 设计也需要在命中率、硬件复杂度、访问延迟和功耗之间折中，而不是简单追求更高的相联度。

4.6 替换算法对 Cache 命中率的影响

当 Cache 发生 Miss 且目标 Set 已满时，需要根据替换算法选择一个 Cache Line 换出。不同替换算法对程序局部性的利用程度不同，因此可能会对 Cache 命中率产生影响。本组实验对比 FIFO 和 LRU 两种替换策略在相同 Cache 配置下的表现。

4.6.1 实验设计

本组实验采用矩阵规模为 $N = 128$ 时生成的 trace.log 作为输入，固定 Cache 块大小为 64B，映射方式为 4 路组相联，分别测试 FIFO 和 LRU 两种替换算法下的 Cache 命中率。

实验固定参数如下：

- 矩阵规模：$N = 128$；
- Cache 块大小：64B；
- 映射方式：4 路组相联；
- 输入文件：$N = 128$ 时由 NEMU 生成的 trace.log。

变化参数为替换算法：

- FIFO：先进先出替换算法；
- LRU：最近最少使用替换算法。

4.6.2 实验结果

Table 4: FIFO 与 LRU 替换算法对命中率的影响

Cache 大小	替换算法	命中率 (%)	Miss 数
16KB	FIFO	49.76	2136344
16KB	LRU	49.75	2136615
32KB	FIFO	49.77	2135968
32KB	LRU	49.76	2136208
64KB	FIFO	97.27	116087
64KB	LRU	97.26	116568
128KB	FIFO	99.85	6174
128KB	LRU	99.83	7200

实验数据绘制如下图:

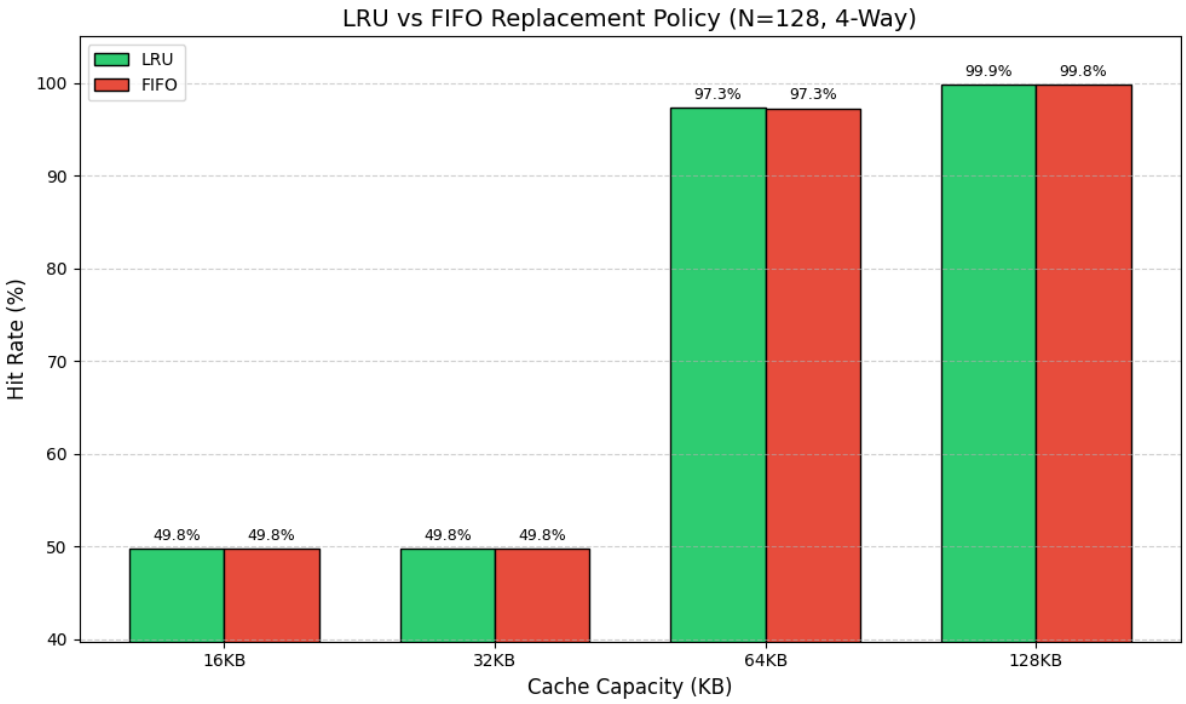

Figure 3: 替换策略对 Cache 命中率的影响

4.6.3 实验现象与分析

从实验结果可以看出，在本组矩阵乘法 trace 下，FIFO 和 LRU 两种替换算法的命中率差异很小，整体表现非常接近。具体而言，在 16KB、32KB、64KB 和 128KB 四种 Cache 容量下，FIFO 的命中率分别为 49.76%、49.77%、97.27% 和 99.85%；LRU 的命中率分别为 49.75%、49.76%、97.26% 和 99.83%。从数值上看，FIFO 在本实验中反而略高于 LRU，但差距均较小。

在 16KB 和 32KB 的低容量阶段，两种算法的命中率都维持在约 49.7%。这说明此时主要瓶颈并不是替换算法选择，而是 Cache 容量不足。由于矩阵规模为 $N = 128$ 时三个

矩阵总大小达到 192KB，而 16KB 和 32KB 的 Cache 远不能容纳主要工作集，因此大量 Miss 属于容量缺失。在这种情况下，即使使用 LRU，也难以通过保留“最近访问的数据”显著改善命中率。

当 Cache 容量增加到 64KB 和 128KB 时，两种算法的命中率都明显提高，说明容量增加消除了大量容量缺失。此时 FIFO 和 LRU 的差距仍然很小，说明该矩阵乘法访问序列在当前配置下对替换策略并不十分敏感。换言之，影响命中率的主要因素是 Cache 容量是否能够覆盖足够大的活跃工作集，而不是 FIFO 与 LRU 之间的细微差别。

从理论上看，LRU 更符合时间局部性原理，因为它倾向于保留最近被访问过的数据块；FIFO 实现更简单，但不关心数据近期是否被访问。然而，本实验数据说明，理论上更复杂的 LRU 并不一定在所有访存序列下都优于 FIFO。对于本实验中的矩阵乘法 trace，FIFO 和 LRU 的结果几乎相同，并且 FIFO 略优。

4.6.4 实验总结

通过 FIFO 和 LRU 的对比可以看出，替换算法对命中率的影响依赖于具体访存序列和 Cache 容量。在本实验中，FIFO 与 LRU 的命中率差距很小，且 FIFO 在各组容量下均略高于 LRU，因此不能简单得出“LRU 一定优于 FIFO”的结论。

更准确地说，LRU 是一种符合时间局部性思想的通用替换策略，在许多存在明显重复访问的程序中可能表现较好；但当程序的主要 Miss 来源是容量不足，或者访问模式使得替换策略难以发挥作用时，LRU 的优势会被削弱。对于本实验的矩阵乘法 trace，Cache 容量对命中率的影响明显大于替换算法本身。

4.7 矩阵规模对 Cache 命中率的影响

为了分析程序工作集大小对 Cache 命中率的影响，本实验进一步改变矩阵乘法程序中的矩阵规模，分别测试 $N = 32$、$N = 64$、$N = 128$ 三种情况下的 Cache 命中率变化。

矩阵规模会直接影响程序运行时的访存工作集大小。当矩阵规模较小时，主要数据可以较容易地被 Cache 容纳；当矩阵规模增大后，程序访问的数据总量迅速增加，Cache 容量不足时会产生更多容量缺失。

4.7.1 实验设计

本组实验固定 Cache 配置，改变矩阵规模。固定参数如下：

- Cache 块大小：64B；
- 映射方式：4 路组相联；
- 替换算法：LRU。

变化参数为矩阵规模：

- $N = 32$；
- $N = 64$；
- $N = 128$。

对于 int 型矩阵而言，每个元素占 4B。因此，单个 $N \times N$ 矩阵大小为：

$$MatrixSize = N \times N \times 4B$$

三个矩阵 A、B、C 的总数据量为：

$$WorkingSet = 3 \times N \times N \times 4B$$

4.7.2 不同矩阵规模下的工作集大小

Table 5: 不同矩阵规模下的数据规模

矩阵规模	单个矩阵大小	三个矩阵总大小
$N = 32$	4KB	12KB
$N = 64$	16KB	48KB
$N = 128$	64KB	192KB

从表中可以看出，随着矩阵规模增大，程序的工作集大小呈平方级增长。当 $N = 32$ 时，三个矩阵总大小仅为 12KB，较容易被 Cache 容纳；当 $N = 128$ 时，三个矩阵总大小达到 192KB，已经明显超过常见的实验 Cache 容量，因此会产生更多容量缺失。

4.7.3 实验结果

Table 6: 矩阵规模对 Cache 命中率的影响

矩阵规模	Cache 大小	命中率 (%)	Miss 数
$N = 32$	32KB	99.70	208
$N = 64$	32KB	99.71	1566
$N = 128$	32KB	49.77	2135968
$N = 32$	64KB	99.70	208
$N = 64$	64KB	99.85	1566
$N = 128$	64KB	97.27	116087

实验数据绘图如下图:

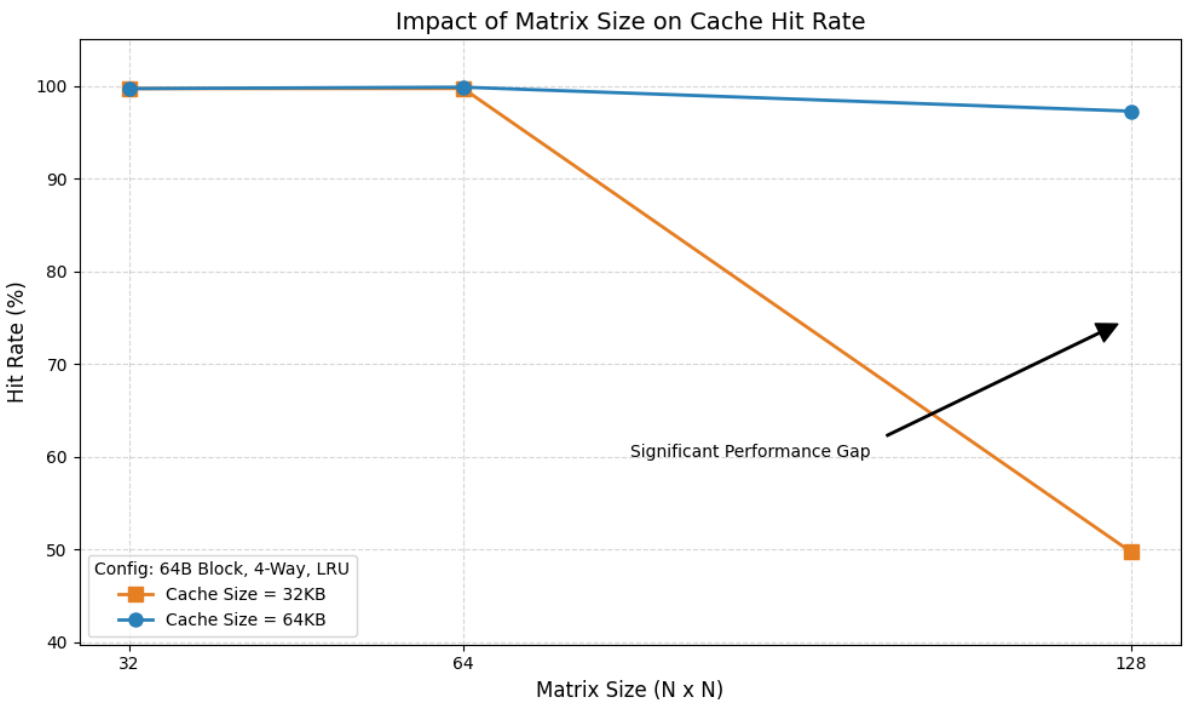

Figure 4: 矩阵规模对 Cache 命中率影响

4.7.4 实验现象与分析

当矩阵规模为 $N = 32$ 时，单个矩阵大小为 4KB，三个矩阵总大小为 12KB。在 Cache 容量为 32KB 和 64KB 时，工作集规模均小于 Cache 容量，因此两种配置下的命中率都达到 99.70%，Miss 数均为 208。这说明在小规模矩阵下，Cache 容量已经足够覆盖主要工作集，继续增大 Cache 容量并不会带来明显收益。

当矩阵规模增大到 $N = 64$ 时，单个矩阵大小为 16KB，三个矩阵总大小为 48KB。此时 32KB Cache 已经小于三个矩阵的总大小，但实验结果显示命中率仍然达到 99.71%；64KB Cache 的命中率为 99.85%，相比 32KB 仅有小幅提升。这说明虽然总数据量有所增加，但程序在实际运行过程中并不是同时频繁访问全部数据，而是存在一定的活跃工作集和局部性，因此 32KB Cache 仍能维持较高命中率。

当矩阵规模进一步增大到 $N = 128$ 时，单个矩阵大小为 64KB，三个矩阵总大小达到 192KB。此时 32KB Cache 已明显无法覆盖主要活跃数据，命中率下降到 49.77%，Miss 数达到 2135968；而 64KB Cache 的命中率达到 97.27%，Miss 数降为 116087。由此可见，真正显著的容量差异出现在 $N = 128$ 的大规模矩阵场景中。

因此，本组实验的关键现象不是“矩阵规模一增大命中率就立即下降”，而是当矩阵规模增长到使活跃工作集明显超过 Cache 容量时，命中率才会发生显著下降。对于 $N = 32$ 和 $N = 64$，局部性仍然能够维持较高命中率；对于 $N = 128$，32KB Cache 出现明显容量瓶颈，而 64KB Cache 则能显著缓解这一问题。

4.7.5 实验总结

通过不同矩阵规模的对比可以看出，矩阵规模会影响程序的工作集大小，但命中率是否显著下降，还取决于活跃工作集与 Cache 容量之间的匹配关系。

在 $N = 32$ 和 $N = 64$ 时，虽然矩阵规模增大，但程序仍能保持较高命中率，说明此时 Cache 容量和程序局部性仍然能够支撑主要访存需求。而在 $N = 128$ 时，32KB Cache 的命中率显著下降到 49.77%，说明容量不足已经成为主要瓶颈；当 Cache 容量增大到 64KB 后，命中率恢复到 97.27%，说明较大的 Cache 能够覆盖更多活跃数据，从而显著减少 Miss。

因此，Cache 性能不仅取决于矩阵总数据量，也取决于程序在某一阶段实际反复访问的活跃数据规模。对于矩阵乘法这类数据密集型程序，当矩阵规模较大时，若不通过循环交换、分块等方法改善访存局部性，仅依靠较小容量的 Cache 难以持续保持高命中率。

4.8 矩阵乘法算法对 Cache 命中率的影响

前面的实验主要从 Cache 参数角度分析了命中率变化，包括 Cache 容量、块大小、相联度、替换算法和矩阵规模等因素。但对于矩阵乘法这类数据密集型程序而言，程序本身的访存模式同样会显著影响 Cache 命中率。因此，本节进一步比较普通三重循环矩阵乘法和分块矩阵乘法在不同 Cache 容量下的命中率差异。

4.8.1 实验设计

本组实验选取矩阵规模为 $N = 128$ 的访存 trace 作为分析对象，比较两种矩阵乘法实现方式：

- 普通 IJK 矩阵乘法：直接使用三重循环计算矩阵乘法；
- 分块矩阵乘法：将矩阵划分为若干小块，使局部计算所需的数据尽可能保留在 Cache 中。

实验固定参数如下：

- 矩阵规模：$N = 128$；
- Cache 块大小：64B；
- 映射方式：4 路组相联；
- 替换算法：LRU；
- Cache 容量：4KB、8KB、16KB、32KB、64KB、128KB。

本实验分别使用普通矩阵乘法生成的 `trace_n128.log` 和分块矩阵乘法生成的 `trace_blocked_n128.log` 作为输入，通过自行编写的 Cache 模拟器统计不同容量下的 Cache 命中率。

4.8.2 普通矩阵乘法的访存特征

普通 IJK 矩阵乘法的核心形式如下：

Listing 2: 普通 IJK 矩阵乘法

```
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        for (int k = 0; k < N; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

在该实现中，矩阵 A 的访问形式为 $A[i][k]$，随着 k 增加，访问的是同一行中的连续元素，因此具有较好的空间局部性。矩阵 $C[i][j]$ 在内层循环中被反复累加，因此具有一定的时间局部性。

但是，矩阵 B 的访问形式为 $B[k][j]$。当 k 增加时，程序访问的是矩阵 B 的同一列，内存地址之间的跨度为一整行。对于 $N = 128$ 的 int 型矩阵，一行大小为：

$$128 \times 4B = 512B$$

这意味着矩阵 B 的连续访问步长为 512B，远大于本实验中的 Cache 块大小 64B。因此，普通 IJK 矩阵乘法难以充分利用矩阵 B 的空间局部性，容易造成大量 Cache Miss。

4.8.3 分块矩阵乘法的优化思想

分块矩阵乘法的核心思想是将原矩阵划分为若干较小的子块，每次只在局部子块上进行计算。其基本形式如下：

Listing 3: 分块矩阵乘法

```
for (int ii = 0; ii < N; ii += BS) {
    for (int jj = 0; jj < N; jj += BS) {
        for (int kk = 0; kk < N; kk += BS) {
            for (int i = ii; i < ii + BS; i++) {
                for (int j = jj; j < jj + BS; j++) {
                    for (int k = kk; k < kk + BS; k++) {
                        C[i][j] += A[i][k] * B[k][j];
                    }
                }
            }
        }
    }
}
```

其中，BS 表示分块大小。与普通 IJK 算法相比，分块算法并没有改变矩阵乘法的数学结果，而是改变了程序访问内存的顺序。通过在一个较小的数据块内集中完成计算，分块算法可以提高数据被重复利用的概率，使矩阵 A、B 和 C 的局部子块尽可能停留在 Cache 中，从而减少容量缺失和替换缺失。

4.8.4 实验结果

Table 7: 相联度对 Cache 命中率的影响

Cache 大小	IJK 算法命中率	分块算法命中率
4KB	49.75	96.69
8KB	49.75	96.69
16KB	49.76	98.99
32KB	49.77	99.74
64KB	97.27	99.77
128	99.85	99.87

实验结果数据绘制如图 5 所示。图中红色曲线表示普通 IJK 矩阵乘法，绿色曲线表示分块矩阵乘法。

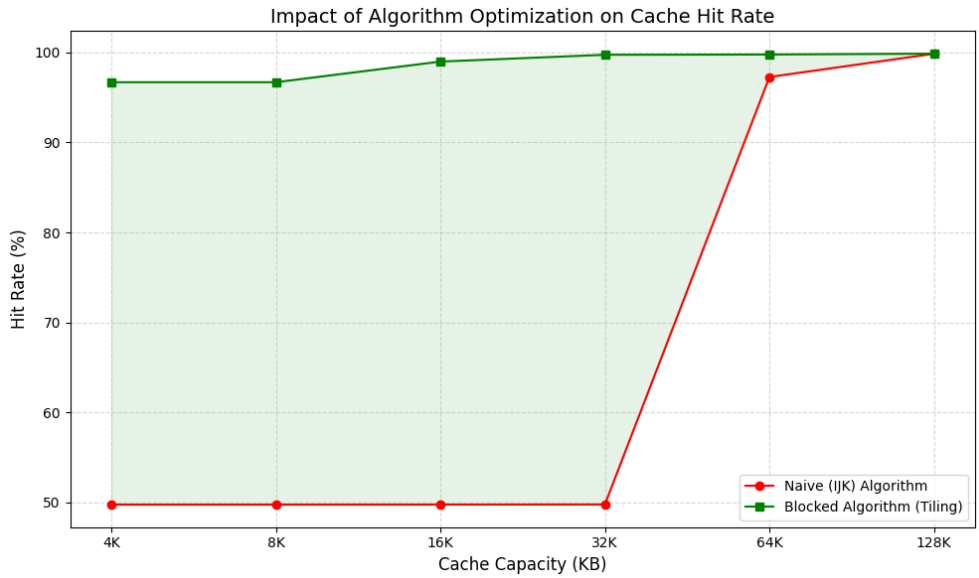

Figure 5: 矩阵乘法算法对 Cache 命中率的影响

从图中可以看出，在 Cache 容量较小时，普通 IJK 算法的命中率长期维持在约 50% 左右；而分块矩阵乘法即使在 4KB、8KB 等较小 Cache 容量下，也能保持较高命中率，约在 96% 以上。随着 Cache 容量增大，分块算法的命中率进一步接近 100%。

当 Cache 容量达到 64KB 及以上时，普通 IJK 算法的命中率也出现明显提升。这与前文 Cache 容量实验中的现象一致：当 Cache 容量达到单个 128×128 int 型矩阵大小附近时，普通算法中的活跃数据可以被更好地缓存，因此命中率发生跃升。

4.8.5 实验现象与分析

本组实验最明显的现象是：在小容量 Cache 下，分块矩阵乘法远优于普通 IJK 矩阵乘法。普通 IJK 算法在 4KB 到 32KB 的 Cache 容量范围内命中率基本维持在约 50%，说明此时 Cache 容量不足以容纳主要活跃数据，同时矩阵 B 的跨行访问导致空间局部性利用较差。

相比之下，分块矩阵乘法通过限制每次计算所涉及的数据范围，使程序在短时间内反复访问同一批矩阵子块。这样一来，已经装入 Cache 的数据块在被替换之前可以被多次使用，从而显著提高时间局部性和空间局部性的利用效率。因此，即使 Cache 容量只有 4KB 或 8KB，分块算法仍然能够维持较高命中率。

从程序访存模式看，普通 IJK 算法的主要问题在于矩阵 $B[k][j]$ 的按列访问。每次访问相邻的 k 值时，地址跨越 512B，远大于 64B 的 Cache 块大小，因此一个 Cache 块被加载后，其中相邻数据很难被立即利用。分块算法则通过固定一小段 kk 范围，使矩阵 B 的局部块在一段时间内被反复访问，从而提高了 Cache 块中数据的利用率。

此外，图中也显示出：随着 Cache 容量增大，普通 IJK 算法和分块算法之间的差距逐渐缩小。当 Cache 容量达到 128KB 时，两种算法的命中率都接近 100%。这说明当 Cache 容量足够大时，硬件容量本身已经能够覆盖大部分活跃工作集，代码访存优化带来的额外收益会减小；而当 Cache 容量较小时，分块优化对命中率的提升最为明显。

4.8.6 实验总结

通过普通 IJK 矩阵乘法和分块矩阵乘法的对比可以看出，Cache 命中率不仅取决于 Cache 容量、块大小、相联度和替换算法，也强烈依赖于程序本身的访存模式。

普通矩阵乘法由于矩阵 B 的按列访问，难以充分利用 Cache 块内的空间局部性，因此在小容量 Cache 下命中率较低。分块矩阵乘法通过限制局部计算的数据范围，使矩阵子块能够在 Cache 中被重复利用，从而显著提高命中率。

因此，对于矩阵乘法这类数据密集型程序，单纯依靠增大 Cache 容量并不是唯一优化方式。通过分块等程序级优化方法改善访存局部性，可以在 Cache 容量较小的情况下显著减少 Cache Miss，提高程序的访存效率。

4.9 香山 Cache 性能计数器统计与对比（选做）

除了使用自行编写的 Cache 模拟器分析 trace.log 外，本实验进一步尝试使用香山仿真环境中的性能计数器统计 Cache 相关行为。该部分作为选做实验，用于从更接近真实处理器微结构的角度观察程序的 Cache 命中与缺失情况，并与软件 Cache 模拟器的结果进行对比分析。

4.9.1 实验原理

前文使用的 NEMU 主要用于执行 RISC-V 程序并导出访存 trace。自行编写的 Cache 模拟器则基于该 trace，以软件方式模拟 Cache 的地址映射、命中检查和替换过程。

相比之下，香山仿真环境能够模拟更接近真实处理器的微结构行为。通过性能计数器，可以统计程序运行过程中的 Load 次数、Load Miss 次数、L3 访问次数以及平均访存延迟等指标。因此，香山性能计数器统计结果不仅反映访存地址序列本身，也包含处理器流水线、乱序执行、MissQueue、MSHR 等硬件机制的影响。

4.9.2 实验流程

首先进入香山目录，生成香山仿真程序：

```
cd /xs-env/XiangShan
make init
make emu CONFIG=MinimalConfig EMU_TRACE=1 -j32
```

随后编译 NEMU，用于差分测试：

```
cd /xs-env/NEMU
make clean
make riscv64-xs-ref_defconfig
make -j
```

最后使用香山仿真器运行 workload，并将性能计数器信息导出到日志文件：

```
./build/emu \
-i /home/infinite/xs-env/nexus-am/apps/hello/build/hello-riscv64-xs.
    bin \
--force-dump-result \
2>&1 | tee hello_perf.log
```

其中，`hello_perf.log` 中包含程序运行过程中的性能计数器输出，后续通过脚本提取 Cache 相关统计项。

4.9.3 性能计数器解析方法

为了从日志文件中提取性能计数器信息，本实验使用 Python 正则表达式匹配 `[PERF]` 开头的记录，并提取模块名、计数器名称和对应数值。核心解析逻辑如下：

Listing 4: 香山性能计数器日志解析脚本

```
import re
import csv
import argparse
from collections import defaultdict

PERF_RE = re.compile(
    r"^\[PERF \]\[time=\s*(\d+)\]\s+([^:]+):\s*([^,]+),\s*(-?\d+)"
)

def parse_perf(path):
    records = []
    with open(path, "r", errors="ignore") as f:
        for line in f:
            m = PERF_RE.match(line.strip())
            if not m:
                continue
            time, module, counter, value = m.groups()
            records.append({
                "time": int(time),
                "module": module.strip(),
                "counter": counter.strip(),
                "value": int(value),
                "full_name": f"{module.strip()}.{counter.strip()}",
            })
    return records

def export_csv(records, out):
```

```python
    with open(out, "w", newline="") as f:
        writer = csv.DictWriter(
            f,
            fieldnames=["time", "module", "counter", "value", "full_name"]
        )
        writer.writeheader()
        writer.writerows(records)

def grep_records(records, keywords):
    kws = [k.lower() for k in keywords]
    return [
        r for r in records
        if any(k in r["full_name"].lower() for k in kws)
    ]

def sum_counter(records, suffix):
    return sum(
        r["value"] for r in records
        if r["full_name"].endswith(suffix)
    )

def print_cache_summary(records):
    print("========== Cache Summary ==========")

    num_loads = sum_counter(records, "dcache.num_loads")
    miss_queue_fire = sum_counter(records, "dcache.miss_queue_fire")
    io_refill = sum_counter(records, "fdpMonitor.io_refill")

    load_req = sum(
        r["value"] for r in records
        if re.search(r"dcache\.ldu_\d+\.load_req$", r["full_name"])
    )
    load_hit = sum(
        r["value"] for r in records
        if re.search(r"dcache\.ldu_\d+\.load_hit$", r["full_name"])
    )
    load_miss = sum(
        r["value"] for r in records
        if re.search(r"dcache\.ldu_\d+\.load_miss$", r["full_name"])
    )

    store_miss_alloc = sum_counter(records, "missQueue.miss_req_stor_allocate")
    load_miss_alloc = sum_counter(records, "missQueue.miss_req_load_allocate")
    miss_req = sum_counter(records, "missQueue.miss_req")
    miss_req_alloc = sum_counter(records, "missQueue.miss_req_allocate")
```

```python

    print(f"DCache num_loads              = {num_loads}")
    print(f"LDU load_req sum              = {load_req}")
    print(f"LDU load_hit sum              = {load_hit}")
    print(f"LDU load_miss sum             = {load_miss}")
    print(f"DCache miss_queue_fire        = {miss_queue_fire}")
    print(f"DCache io_refill              = {io_refill}")
    print(f"MissQueue miss_req            = {miss_req}")
    print(f"MissQueue miss_req_allocate   = {miss_req_alloc}")
    print(f"MissQueue load miss allocate  = {load_miss_alloc}")
    print(f"MissQueue store miss allocate = {store_miss_alloc}")

    if load_req:
        print(f"Strict load miss rate         = {load_miss / load_req * 100:.6f}%")
    if num_loads:
        print(f"Approx DCache miss rate       = {miss_queue_fire / num_loads * 100:.6f}%")

    l3_nr = sum_counter(records, "busPMU.L3_Mem_nrRecord_all")
    l3_lat = sum_counter(records, "busPMU.L3_Mem_latencySum_all")
    if l3_nr:
        print(f"L3/Mem records                = {l3_nr}")
        print(f"L3/Mem latency sum            = {l3_lat}")
        print(f"L3/Mem avg latency            = {l3_lat / l3_nr:.6f} cycles")

    icache_refill = sum_counter(records, "icache.metaArray.meta_refill_num")
    icache_miss_bubble = sum_counter(records, "icache.mainPipe.icache_bubble_s2_miss")
    print(f"ICache meta_refill_num        = {icache_refill}")
    print(f"ICache bubble_s2_miss         = {icache_miss_bubble}")

def print_top(records, n=30, keywords=None):
    selected = records
    if keywords:
        selected = grep_records(records, keywords)
    selected = sorted(selected, key=lambda x: abs(x["value"]), reverse=True)

    print(f"========== Top {n} Counters ==========")
    for r in selected[:n]:
        print(f"{r['value']:>12}  {r['full_name']}")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("logfile")
    ap.add_argument("--csv", default="perf.csv")
```

```
    ap.add_argument("--grep", nargs="*", default=None)
    ap.add_argument("--top", type=int, default=30)
    args = ap.parse_args()

    records = parse_perf(args.logfile)

    print(f"Parsed PERF records: {len(records)}")
    export_csv(records, args.csv)
    print(f"CSV exported to: {args.csv}")

    print_cache_summary(records)

    if args.grep:
        print_top(records, args.top, args.grep)
    else:
        print_top(records, args.top)

if __name__ == "__main__":
    main()
```

通过该脚本可以从香山仿真日志中提取 Load 次数、Load Miss 次数、L3 访问次数和访存延迟等信息。

4.9.4 实验结果

本组选做实验分别统计了矩阵规模为 $N = 32$、$N = 64$、$N = 128$ 时的 DCache 访存行为，结果如下：

Table 8: 香山性能计数器统计的 DCache 访存行为

矩阵规模	Load 次数	Load Miss	Miss 率 (%)	L3 访问次数	平均延迟
$N = 32$	72,698	0	0.00	212	17
$N = 64$	573,575	3,330	0.58	786	17
$N = 128$	9,340,086	6,827,353	73.10	3092	17

根据 Miss 率可以进一步得到命中率：

$$HitRate = 1 - MissRate$$

因此，当 $N = 32$ 时，命中率接近 100%；当 $N = 64$ 时，命中率约为 99.42%；当 $N = 128$ 时，命中率下降到约 26.90%。

Table 9: 香山性能计数器统计的 Cache 命中率

矩阵规模	Miss 率 (%)	命中率 (%)
$N = 32$	0.00	100.00
$N = 64$	0.58	99.42
$N = 128$	73.10	26.90

4.9.5 结果分析

从实验结果可以看出，随着矩阵规模增大，香山性能计数器统计得到的 Cache 命中率呈明显下降趋势。

当 $N = 32$ 时，矩阵数据规模较小，整个工作集可以较好地驻留在 Cache 中，因此 Load Miss 数为 0，命中率接近 100%。当 $N = 64$ 时，数据规模有所增加，但程序仍然具有较好的时间局部性和空间局部性，同时香山处理器具有多级 Cache 结构，因此整体 Miss 率仍然较低，仅为 0.58%。

当 $N = 128$ 时，三个矩阵的总数据规模达到 $192KB$，已经明显超过较小 Cache 的可容纳范围。此时程序运行过程中会产生大量 Cache 替换，Load Miss 数显著增加，Miss 率达到 73.10%，说明 Cache 容量和访存局部性已经成为程序性能的重要瓶颈。

值得注意的是，$N = 128$ 时 Load Miss 数远大于 L3 访问次数。这说明香山处理器内部并不是将每一次 Load Miss 都直接转化为一次下级存储访问，而是可能通过 MissQueue、MSHR 等机制对多个 Miss 请求进行合并，从而减少对更低层存储结构的实际访问次数。

4.9.6 与软件 Cache 模拟器结果的对比

自行编写的 Cache 模拟器是基于 NEMU 输出的访存 trace 进行顺序模拟，通常只模拟单级 Cache，并按照给定的 Cache 容量、块大小、相联度和替换算法判断 Hit/Miss。因此，该模拟器更适合用于分析 Cache 参数变化对命中率的影响。

香山性能计数器则反映更接近真实乱序处理器的运行行为。其统计结果不仅受访存地址序列影响，还受到流水线执行、Load 重放、投机执行、Cache Bank 冲突、MissQueue 合并以及多级 Cache 结构等因素影响。因此，香山统计结果与软件 Cache 模拟器结果在绝对数值上不一定完全一致。

例如，在矩阵规模较小时，两者都表现出较高的 Cache 命中率；而当矩阵规模增大到 $N = 128$ 时，两者都显示 Cache 性能显著下降。这说明二者在趋势上具有一致性：随着工作集规模增大，Cache 容量压力上升，Cache Miss 增多。但由于两者建模层次不同，不能简单要求二者的命中率数值完全相同。

4.9.7 选做实验总结

通过香山性能计数器统计可以看出，真实处理器中的 Cache 行为比单级软件 Cache 模拟器更加复杂。软件模拟器便于控制变量，适合分析 Cache 大小、块大小、相联度和替换算法等参数的影响；香山性能计数器则能够体现更真实的处理器微结构行为，包括多级 Cache、Miss 合并和访存延迟等因素。

因此，将自行编写的 Cache 模拟器与香山性能计数器结果结合起来，可以从“理想化模型”和“真实微结构”两个层面理解 Cache 行为。这也说明，Cache 命中率不仅取决于程序访存模式和 Cache 参数，还受到处理器具体实现机制的影响。

5 实验总结

本次实验围绕 NEMU 模拟器和 Cache 行为仿真展开，主要完成了从程序运行、访存 trace 获取，到 Cache 模拟器设计、正确性验证以及不同 Cache 参数分析的完整实验流程。

首先，本实验通过香山仿真环境中的 AM 编译矩阵乘法 workload，并使用 NEMU 模拟器运行生成的二进制程序。通过开启内存访问日志功能，实验获得了程序运行过程中的内存访问序列 trace.log。相比直接在真实硬件上观察程序访存行为，NEMU 提供了一

个更加可控、可重复、便于分析的实验环境，使得程序执行过程中的访存地址和读写行为能够被记录下来，为后续 Cache 行为分析提供了基础数据。

其次，本实验自行实现了 Cache 模拟器。该模拟器以 NEMU 生成的 trace.log 为输入，通过软件方式模拟 Cache 的地址映射、Tag 匹配、命中判断和替换过程。实验中实现了直接映射、组相联映射和全相联映射等不同映射方式，并支持 FIFO 和 LRU 两种替换算法。通过 micro-benchmarking 方法，本实验构造了若干组简单访存序列，并将人工推导的 Hit/Miss 结果与模拟器输出结果进行对比，从而验证了模拟器核心逻辑的正确性。

在 Cache 参数分析方面，实验结果表明，Cache 容量对命中率具有明显影响。对于矩阵规模为 $N = 128$ 的普通矩阵乘法程序，当 Cache 容量较小时，命中率长期维持在较低水平；当 Cache 容量达到单个矩阵大小附近时，命中率出现明显跃升。这说明程序工作集大小与 Cache 容量之间的匹配关系，是影响 Cache 性能的重要因素。

同时，实验也分析了 Cache 块大小、相联度和替换算法的影响。Cache 块大小主要影响程序对空间局部性的利用程度；相联度主要影响冲突缺失。理论上，提高相联度可以减少多个活跃数据块竞争同一 Cache 位置所造成的冲突 Miss；但本实验中 2 路组相联命中率最高，继续提高到 4 路、8 路或全相联并没有带来单调提升，说明相联度的效果还会受到组数减少、地址分布和替换行为的共同影响。替换算法则影响 Cache 在容量有限时保留哪些数据块。在替换算法方面，LRU 从原理上更符合时间局部性思想，但本实验数据中 FIFO 与 LRU 的命中率非常接近，且 FIFO 在各组 Cache 容量下略高于 LRU。这说明替换算法的实际效果依赖于具体访存序列和 Cache 容量，当主要 Miss 来源是容量不足或程序访问模式对替换策略不敏感时，LRU 不一定表现出明显优势。但当程序 Miss 主要由容量不足或顺序扫描访问造成时，单纯改进替换算法带来的提升也会受到限制。

此外，通过不同矩阵规模的对比可以看出，随着矩阵规模增大，程序的工作集大小迅速增加，Cache 容量压力随之上升。对于普通三重循环矩阵乘法而言，矩阵 A 的按行访问具有较好的空间局部性，矩阵 C 的累加具有一定时间局部性，而矩阵 $B[k][j]$ 的按列访问会造成较大的访存步长，难以充分利用 Cache 块内的空间局部性。因此，非优化矩阵乘法的 Cache 性能不仅受硬件参数影响，也明显受代码访存模式影响。

在选做实验中，本实验进一步使用香山性能计数器统计了不同矩阵规模下的 DCache 访存行为。实验结果表明，随着矩阵规模从 $N = 32$ 增大到 $N = 128$，香山统计得到的 Cache 命中率从接近 100% 下降到约 26.90%，说明在真实处理器微结构下，数据规模增大同样会显著增加 Cache 压力。与自行编写的 Cache 模拟器相比，香山性能计数器结果包含乱序执行、Load 重放、MissQueue 合并、多级 Cache 等硬件机制影响，因此二者在绝对数值上存在差异是合理的，但在“矩阵规模增大导致 Cache 性能下降”这一趋势上保持一致。

综上，本次实验加深了对 Cache 工作机制的理解。Cache 命中率并不是由单一因素决定的，而是由 Cache 容量、块大小、相联度、替换算法以及程序访存模式共同决定。对于数据密集型程序，除了调整 Cache 参数外，从程序层面改善局部性，例如循环交换、分块矩阵乘法等方法，也能够有效降低 Cache Miss，提高程序运行效率。

A Cache 模拟器代码

```python
#!/usr/bin/env python3
"""A small configurable cache simulator for NEMU memory traces.

The simulator supports direct mapped, N-way set associative, and fully
associative caches with FIFO or LRU replacement.  It parses log lines such as:

    load: addr = 0x100000000, len = 2, data = 0x93
    write: addr = 0x100003780, len = 4, data = 0x0

An access that spans multiple cache blocks is counted as one probe per block,
which matches the hardware event that more than one cache line may be touched.
"""

from __future__ import annotations

import argparse
import csv
import re
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable, Iterator, Literal, TextIO


Policy = Literal["lru", "fifo"]

TRACE_RE = re.compile(
    r"\b(?P<op>load|write):\s+addr\s+=\s+0x(?P<addr>[0-9a-fA-F]+),\s+len\s+=\s+(?P<size>\d+)"
)


@dataclass
class CacheLine:
    valid: bool = False
    tag: int = -1
    inserted_at: int = 0
    last_used_at: int = 0


@dataclass
class CacheStats:
    accesses: int = 0
    hits: int = 0
```

```python
    misses: int = 0
    reads: int = 0
    writes: int = 0

    @property
    def hit_rate(self) -> float:
        return self.hits / self.accesses if self.accesses else 0.0

    @property
    def miss_rate(self) -> float:
        return self.misses / self.accesses if self.accesses else 0.0

    def as_row(self) -> dict[str, int | float]:
        return {
            "accesses": self.accesses,
            "hits": self.hits,
            "misses": self.misses,
            "hit_rate": self.hit_rate,
            "miss_rate": self.miss_rate,
            "reads": self.reads,
            "writes": self.writes,
        }


class CacheSimulator:
    def __init__(
        self,
        cache_size: int,
        block_size: int,
        associativity: int | str,
        policy: Policy = "lru",
        write_allocate: bool = True,
    ) -> None:
        if block_size <= 0 or cache_size <= 0:
            raise ValueError("cache_size and block_size must be positive")
        if cache_size % block_size != 0:
            raise ValueError("cache_size must be a multiple of block_size")
        if policy not in ("lru", "fifo"):
            raise ValueError("policy must be either 'lru' or 'fifo'")

        self.cache_size = cache_size
        self.block_size = block_size
        self.line_count = cache_size // block_size
        self.policy: Policy = policy
        self.write_allocate = write_allocate

        if associativity == "full":
```

```
            self.associativity = self.line_count
        else:
            self.associativity = int(associativity)
        if self.associativity <= 0:
            raise ValueError("associativity must be positive")
        if self.line_count % self.associativity != 0:
            raise ValueError("line_count must be a multiple of associativity")

        self.set_count = self.line_count // self.associativity
        self.sets = [
            [CacheLine() for _ in range(self.associativity)] for _ in range(self.set_count)
        ]
        self.clock = 0
        self.stats = CacheStats()

    @property
    def mapping_name(self) -> str:
        if self.associativity == 1:
            return "direct"
        if self.associativity == self.line_count:
            return "fully_associative"
        return f"{self.associativity}_way"

    def _decode(self, addr: int) -> tuple[int, int]:
        block_addr = addr // self.block_size
        set_index = block_addr % self.set_count
        tag = block_addr // self.set_count
        return set_index, tag

    def _select_victim(self, lines: list[CacheLine]) -> CacheLine:
        for line in lines:
            if not line.valid:
                return line
        if self.policy == "fifo":
            return min(lines, key=lambda line: line.inserted_at)
        return min(lines, key=lambda line: line.last_used_at)

    def access_block(self, addr: int, is_write: bool = False) -> bool:
        self.clock += 1
        self.stats.accesses += 1
        if is_write:
            self.stats.writes += 1
        else:
            self.stats.reads += 1

        set_index, tag = self._decode(addr)
```

```python
        lines = self.sets[set_index]
        for line in lines:
            if line.valid and line.tag == tag:
                self.stats.hits += 1
                line.last_used_at = self.clock
                return True

        self.stats.misses += 1
        if (not is_write) or self.write_allocate:
            victim = self._select_victim(lines)
            victim.valid = True
            victim.tag = tag
            victim.inserted_at = self.clock
            victim.last_used_at = self.clock
        return False

    def access(self, addr: int, size: int = 1, is_write: bool = False
        ) -> list[bool]:
        if size <= 0:
            return []
        start_block = addr // self.block_size
        end_block = (addr + size - 1) // self.block_size
        return [
            self.access_block(block * self.block_size, is_write)
            for block in range(start_block, end_block + 1)
        ]

    def run(self, accesses: Iterable[tuple[str, int, int]]) ->
        CacheStats:
        for op, addr, size in accesses:
            self.access(addr, size, is_write=(op == "write"))
        return self.stats


def parse_size(value: str) -> int:
    text = value.strip().lower()
    multipliers = {
        "b": 1,
        "k": 1024,
        "kb": 1024,
        "m": 1024 * 1024,
        "mb": 1024 * 1024,
    }
    for suffix in ("kb", "mb", "k", "m", "b"):
        if text.endswith(suffix):
            return int(text[: -len(suffix)]) * multipliers[suffix]
    return int(text)


```

```python
def parse_trace(handle: TextIO) -> Iterator[tuple[str, int, int]]:
    for line in handle:
        match = TRACE_RE.search(line)
        if match:
            op = "write" if match.group("op") == "write" else "load"
            yield op, int(match.group("addr"), 16), int(match.group("size"))


def iter_trace_file(path: Path) -> Iterator[tuple[str, int, int]]:
    with path.open("r", encoding="utf-8", errors="ignore") as handle:
        yield from parse_trace(handle)


def simulate_file(
    trace_path: Path,
    cache_size: int,
    block_size: int,
    associativity: int | str,
    policy: Policy,
    write_allocate: bool = True,
) -> tuple[CacheSimulator, CacheStats]:
    sim = CacheSimulator(
        cache_size=cache_size,
        block_size=block_size,
        associativity=associativity,
        policy=policy,
        write_allocate=write_allocate,
    )
    return sim, sim.run(iter_trace_file(trace_path))


def write_csv(path: Path, rows: list[dict[str, object]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    fieldnames = list(rows[0].keys()) if rows else []
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)


def self_test() -> None:
    # Direct mapped cache: blocks 0 and 2 conflict in set 0, block 1 stays in set 1.
    direct = CacheSimulator(cache_size=8, block_size=4, associativity=1, policy="lru")
    direct.run(("load", addr, 1) for addr in [0, 4, 8, 0, 4, 8])
    assert (direct.stats.hits, direct.stats.misses) == (1, 5), direct.stats
```

```python

    # Same fully associative cache, different victim choice for FIFO and LRU.
    seq = [0, 4, 0, 8, 4, 0]
    lru = CacheSimulator(cache_size=8, block_size=4, associativity="full", policy="lru")
    fifo = CacheSimulator(cache_size=8, block_size=4, associativity="full", policy="fifo")
    lru.run(("load", addr, 1) for addr in seq)
    fifo.run(("load", addr, 1) for addr in seq)
    assert (lru.stats.hits, lru.stats.misses) == (1, 5), lru.stats
    assert (fifo.stats.hits, fifo.stats.misses) == (2, 4), fifo.stats

    # A multi-byte access crossing a block boundary should touch two cache lines.
    cross = CacheSimulator(cache_size=16, block_size=4, associativity=1)
    cross.run([("load", 2, 4), ("load", 3, 1)])
    assert (cross.stats.accesses, cross.stats.hits, cross.stats.misses) == (3, 1, 2), cross.stats


def build_arg_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="Cache simulator for NEMU trace.log files")
    parser.add_argument("trace", nargs="?", type=Path, help="trace.log path")
    parser.add_argument("--cache-size", default="4KB", help="total cache size, e.g. 4KB")
    parser.add_argument("--block-size", default="32B", help="cache block size, e.g. 32B")
    parser.add_argument(
        "--assoc",
        default="1",
        help="'1' for direct mapped, N for N-way set associative, or 'full'",
    )
    parser.add_argument("--policy", choices=["lru", "fifo"], default="lru")
    parser.add_argument(
        "--no-write-allocate",
        action="store_true",
        help="do not allocate a line on write misses",
    )
    parser.add_argument("--self-test", action="store_true", help="run micro-benchmark checks")
    return parser


```

```python
def main() -> None:
    parser = build_arg_parser()
    args = parser.parse_args()
    if args.self_test:
        self_test()
        print("self-test passed")
        if args.trace is None:
            return
    if args.trace is None:
        parser.error("trace path is required unless --self-test is used alone")

    assoc: int | str = "full" if args.assoc.lower() == "full" else int(args.assoc)
    sim, stats = simulate_file(
        trace_path=args.trace,
        cache_size=parse_size(args.cache_size),
        block_size=parse_size(args.block_size),
        associativity=assoc,
        policy=args.policy,
        write_allocate=not args.no_write_allocate,
    )
    print(
        f"mapping={sim.mapping_name}, policy={args.policy}, "
        f"cache_size={sim.cache_size}, block_size={sim.block_size}, "
        f"accesses={stats.accesses}, hits={stats.hits}, misses={stats.misses}, "
        f"hit_rate={stats.hit_rate:.6f}, miss_rate={stats.miss_rate:.6f}"
    )


if __name__ == "__main__":
    main()
```

B 分析相联度对 Cache 命中率的影响代码

```python
import csv
from pathlib import Path
from cache_simulator_pro import CacheSimulator, iter_trace_file

def run_assoc_analysis():
    trace_path = Path("traces/trace_n128.log")
    if not trace_path.exists():
        print(f"Error: {trace_path} not found.")
        return

```

```python
    # 固定参数
    CAP_KB = 64# 固定容量为 32KB 以观察冲突改善
    BLOCK_SIZE = 64  # 实验要求: 64B
    POLICY = 'lru'   # 实验要求: LRU

    # 变量: 相联度
    assoc_list = [1, 2, 4, 8, 'full']

    results = []

    print(f"Starting Associativity Analysis (Cap={CAP_KB}KB, Block={BLOCK_SIZE}B)...")

    print("Reading trace into memory...")
    all_accesses = list(iter_trace_file(trace_path))

    for assoc in assoc_list:
        # 初始化模拟器
        sim = CacheSimulator(
            cache_size=CAP_KB * 1024,
            block_size=BLOCK_SIZE,
            associativity=assoc,
            policy=POLICY
        )

        sim.run(all_accesses)
        stats = sim.stats

        # 记录展示名
        display_name = "Full" if assoc == 'full' else f"{assoc}-Way"

        results.append({
            'Associativity': display_name,
            'Hit_Rate': stats.hit_rate * 100,
            'Misses': stats.misses,
            'Accesses': stats.accesses
        })
        print(f"  Assoc: {display_name:>6} | Hit Rate: {stats.hit_rate*100:.2f}%")

    # 保存结果
    output_file = Path('associativity_study.csv')
    with output_file.open('w', newline='') as f:
        fieldnames = ['Associativity', 'Hit_Rate', 'Misses', 'Accesses']
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(results)

```

```
    print(f"\nAnalysis complete. Data saved to {output_file}")

if __name__ == "__main__":
    run_assoc_analysis()
```

C 分析替换算法对 Cache 命中率的影响代码

```
import csv
from pathlib import Path
from cache_simulator_pro import CacheSimulator, iter_trace_file

def run_replacement_analysis():
    trace_path = Path("traces/trace_n128.log")
    if not trace_path.exists():
        print(f"Error: {trace_path} not found.")
        return

    # 固定参数
    BLOCK_SIZE = 64
    ASSOC = 4

    # 变量
    capacities_kb = [16, 32, 64,128]
    policies = ['lru', 'fifo']

    results = []

    print("Starting Replacement Policy Analysis (N=128, 4-Way)...")

    print("Reading trace into memory...")
    all_accesses = list(iter_trace_file(trace_path))

    for cap_kb in capacities_kb:
        print(f"\nTesting Capacity: {cap_kb}KB")
        for poly in policies:
            sim = CacheSimulator(
                cache_size=cap_kb * 1024,
                block_size=BLOCK_SIZE,
                associativity=ASSOC,
                policy=poly
            )

            sim.run(all_accesses)
            stats = sim.stats

            results.append({
                'Capacity_KB': cap_kb,
```

```
                'Policy': poly.upper(),
                'Hit_Rate': stats.hit_rate * 100,
                'Misses': stats.misses
            })
            print(f"  Policy: {poly.upper():<4} | Hit Rate: {stats.hit_rate*100:.2f}%")

    # 保存结果
    output_file = Path('replacement_study.csv')
    with output_file.open('w', newline='') as f:
        fieldnames = ['Capacity_KB', 'Policy', 'Hit_Rate', 'Misses']
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(results)

    print(f"\nAnalysis complete. Data saved to {output_file}")

if __name__ == "__main__":
    run_replacement_analysis()
```

D 分析矩阵规模对 Cache 命中率的影响代码

```
import csv
from pathlib import Path
from cache_simulator_pro import CacheSimulator, iter_trace_file

def run_matrix_size_analysis():
    # 实验变量
    matrix_sizes = [32, 64, 128]
    capacities_kb = [32, 64]

    # 固定参数
    BLOCK_SIZE = 64
    ASSOC = 4
    POLICY = 'lru'

    results = []

    print("Starting Matrix Size vs Cache Capacity Analysis...")

    for n in matrix_sizes:
        trace_path = Path(f"traces/trace_n{n}.log")
        if not trace_path.exists():
            print(f"Warning: {trace_path} not found, skipping N={n}")
            continue

        print(f"\nProcessing Matrix Size N={n}...")
```

```
        # 为了加速，每个 N 只读一次 trace
        all_accesses = list(iter_trace_file(trace_path))

        for cap_kb in capacities_kb:
            sim = CacheSimulator(
                cache_size=cap_kb * 1024,
                block_size=BLOCK_SIZE,
                associativity=ASSOC,
                policy=POLICY
            )

            sim.run(all_accesses)
            stats = sim.stats

            results.append({
                'Matrix_Size': n,
                'Cache_Capacity_KB': cap_kb,
                'Hit_Rate': stats.hit_rate * 100,
                'Misses': stats.misses
            })
            print(f"  Cache: {cap_kb}KB | Hit Rate: {stats.hit_rate
               *100:.2f}%")

    # 保存结果
    output_file = Path('matrix_size_study.csv')
    with output_file.open('w', newline='') as f:
        fieldnames = ['Matrix_Size', 'Cache_Capacity_KB', 'Hit_Rate',
            'Misses']
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(results)

    print(f"\nAnalysis complete. Data saved to {output_file}")

if __name__ == "__main__":
    run_matrix_size_analysis()
```

E 分析矩阵乘法算法对 Cache 命中率的影响代码

```
import csv
from pathlib import Path
from cache_simulator_pro import CacheSimulator, iter_trace_file

def run_algo_comparison():
    # 定义两个 Trace 的路径
    trace_naive = Path("traces/trace_n128.log")          # 普通 IJK 算
       法
```

```python
    trace_blocked = Path("traces/trace_blocked_n128.log") # 分块算法

    if not trace_naive.exists() or not trace_blocked.exists():
        print("Error: Ensure both trace_n128.log and trace_blocked_n128.log exist.")
        return

    # 固定参数：针对矩阵乘法，通常用 4路/8路组相联
    BLOCK_SIZE = 64
    ASSOC = 4
    POLICY = 'lru'

    # 遍历 Cache 容量
    capacities_kb = [4, 8, 16, 32, 64, 128]

    results = []

    # 预加载两个 Trace
    print("Loading Naive Trace...")
    naive_accesses = list(iter_trace_file(trace_naive))
    print("Loading Blocked Trace...")
    blocked_accesses = list(iter_trace_file(trace_blocked))

    for cap_kb in capacities_kb:
        print(f"Simulating Capacity: {cap_kb}KB...")

        # 1. 模拟普通算法
        sim_n = CacheSimulator(cap_kb*1024, BLOCK_SIZE, ASSOC, POLICY)
        sim_n.run(naive_accesses)

        # 2. 模拟分块算法
        sim_b = CacheSimulator(cap_kb*1024, BLOCK_SIZE, ASSOC, POLICY)
        sim_b.run(blocked_accesses)

        results.append({
            'Capacity_KB': cap_kb,
            'Naive_Hit_Rate': sim_n.stats.hit_rate * 100,
            'Blocked_Hit_Rate': sim_b.stats.hit_rate * 100
        })

    # 保存 CSV
    output_file = Path('algorithm_comparison.csv')
    with output_file.open('w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=['Capacity_KB', 'Naive_Hit_Rate', 'Blocked_Hit_Rate'])
        writer.writeheader()
        writer.writerows(results)
```

```python

    print(f"Data saved to {output_file}")

if __name__ == "__main__":
    run_algo_comparison()
```